From Diagnosis to Cure: Decomposing the Tile-SPMD Abstraction Regret on Irregular Automata

ANONYMOUS AUTHOR(S)

Tile-based GPU DSLs (Triton) trail hand-written CUDA by ∼10× on irregular finite-automata workloads. Prior work attributed this "abstraction regret" to the execution *paradigm* (tile/SPMD vs. thread-SIMT) rather than abstraction height. We turn that diagnosis into an anatomy and a cure: we decompose the regret into three independently measured components and identify, to the instruction level, the single missing primitive. On a work-efficient NFA worklist the 10× anchor decomposes as **(A)** a launch-configuration artifact (∼3.4×, default `num_warps`), **(B)** a lane-packing-recoverable component (warp-redundant scalar execution), and **(C)** an irreducible ∼2× residual. Component C is *not* instruction count, occupancy, or memory bandwidth: at matched occupancy and *fewer* warp-instructions than CUDA, and sitting below both the issue and bandwidth roofline ceilings, a per-lane Triton worklist still spends 15.3× more cycles in the dependent-load stall and issues at 9.9% vs. 41% of peak—it is latency-bound. The root cause is abstraction-denied intra-warp memory-level parallelism: a CUDA warp's lanes issue independent in-flight loads that overlap, whereas a lock-step tile serializes the dependent next-state load. We show the residual is *regime-dependent*: for the memory-bound DFA it closes (lane-packed Triton matches CUDA, 1.05×, once the table spills to DRAM). We specify the missing IR primitive—a `scalar_program` region that lowers each lane to an independent instruction stream—and *implement its lowering*: the same idiomatic per-lane source, lowered to the thread model, runs 4.2× faster than the tile lowering and matches or exceeds hand-written CUDA, closing the residual by construction. The phenomenon *generalizes*, including to the ML kernels tile DSLs target: across eight oracle-gated irregular workloads the tile-vs-thread regret is created by per-step *scalar control*—not memory irregularity or dependent loads—validated by a graph pointer-chase negative control (1.00×), confirmed on MoE top-k routing (2.36×), and with a correct *sign-flip* negative on dense ragged attention (0.64×, the tile *wins*)—which is why Triton excels at flash-attention yet collapses on automata. Finally we take the cure into the real compiler. A TritonGPU pass detects the lock-step region in `libtriton`, and a proof shows the in-tile-IR lowering is *structurally impossible* (naming the missing per-lane loop/exit primitive)—so, since the tile IR cannot express it, we **build the cure below it**: a TritonGPU→LLVM pass that redirects the lock-step latch to the per-lane predicate, drops the per-iteration cross-lane reduce, and reconverges with a warp barrier. It is oracle-correct and 4.15× (39× fewer issued instructions; 2.5–7.3× across trip distributions), recovering the residual between the in-IR reduce-hoist (1.55×) and the thread bound (5.64×). Every kernel is correctness-gated against a CPU oracle; every number traces to a versioned CSV.

CCS Concepts: • **Software and its engineering** → **Compilers**; *Parallel programming languages*; • **Computer systems organization** → *Single instruction, multiple data.*

Additional Key Words and Phrases: GPU compilers, tile DSLs, Triton, SIMT, warp-level parallelism, irregular parallelism, per-lane control flow

1 Introduction and Contributions

Prior work [3] shows that on irregular automata the Triton→CUDA gap follows the execution paradigm, not the abstraction height. It leaves open *why*, mechanistically, and *what* would close it. This paper answers both. Our contributions:

(1) A **falsifiable decomposition** of the tile-SPMD automata regret into artifact, recoverable, and irreducible, each measured and Nsight-attributed (Sec. 3, Fig. 1).

2026. ACM 1544-3973/2026/7-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

(2) Identification of the irreducible residual as **abstraction-denied intra-warp latency hiding**—not redundancy, occupancy, instruction count, or bandwidth—*measured*: fewer warp-instructions than CUDA, same occupancy, below both roofline ceilings, yet 15.3× the dependent-load stall and 9.9% vs. 41% issue activity (Sec. 3.4).
(3) A demonstration that the residual is **regime-dependent** (latency-bound NFA vs. memory-bound DFA), unifying both faces through one latency mechanism (Sec. 4).
(4) A **launch-configuration finding** (default `num_warps=4` inflates the worklist regret ∼3.4×), a methodological caution for DSL benchmarking (Sec. 3.2).
(5) The **missing-primitive design** + a bound on its payoff + the thread-model existence proof, distinguished from Gluon's per-thread *layout* (Sec. 5).
(6) **The cure, implemented**: we lower the same idiomatic per-lane source to the thread model and measure that it runs 4.2× faster than the tile lowering and matches/exceeds hand-CUDA, closing the residual by construction (Sec. 5.1, Fig. 6).
(7) **In the real compiler** (Sec. 5.2): a TritonGPU MLIR pass that compiles into `libtriton` and *detects* the lock-step region; a proof that the in-IR lowering is *structurally blocked* (`scf.condition` is a single `i1`; the tensors are already `sizePerThread=1`), naming the missing IR primitive (a per-lane sub-tile loop/exit op); and an *automatic* detect-and-lower selector that routes the detected region to the thread cure for 3.9× (negative-control kernels stay on the tile path); a real in-compiler rewrite built into `libtriton` (reduce-hoist, 1.55×, oracle-correct); **and the full per-lane retirement cure, built below the tile IR**—a TritonGPU→LLVM pass wired into `make_llir`, oracle-correct and 4.15× (39× fewer issued instructions, 2.5–7.3× across trip distributions), closing the residual by construction.
(8) A **generality law** across eight oracle-gated irregular workloads (Sec. 5.3, Fig. 7), including the ML kernels tile DSLs target: regret is created by per-step *scalar control*—not memory irregularity—validated by a pointer-chase negative control (1.00×), confirmed on MoE top-k routing (2.36×), and with the correct *sign flip* predicted on dense ragged attention (0.64×, the tile wins).

2 Background and Method

NFA simulation (a work-efficient worklist iterating the active set via `ffs` over a bit-mask) is control-flow/latency-bound; DFA simulation (one dependent table gather `trans[cur*256+sym]` per symbol) is memory-bound. CUDA and NVIDIA Warp [17] are thread-SIMT (one thread per string); Triton [23] is tile/SPMD (a program operates on tiles). Correctness gates speed: every kernel/config is validated bit-for-bit against a CPU oracle before any throughput is reported. We report medians over warmup+9 reps at a GPU-saturating batch [11] (timings on GPUs are non-Gaussian, so we avoid mean±std and best-of-N), and sweep batch because the lane-packing benefit is occupancy-gated. Every mechanism claim is backed by Nsight Compute, not just wall-clock: the issue-stall composition (the `long_scoreboard` reason isolates dependent-load waits), achieved occupancy, issue activity, warp- and thread-instruction counts, and DRAM/L2 traffic. We report negative and partial results and correct three of our own intermediate hypotheses (Secs. 3, 3.4, 5.3). Hardware: RTX 4070 (sm_89, 46 SMs), Triton 3.5.1, torch 2.9.1+cu128, CUDA 13.3 / driver 580; the in-`libtriton` compiler pass (§5.2) uses a from-source Triton 3.8.

3 The Decomposition (NFA worklist)

3.1 The anchor

A work-efficient register-resident worklist ($\leq$ 64 states, batch 4096, 12 configs): Triton 22 Gbps vs. CUDA 227 Gbps = 10.1× **median** (9.4–12.3×).

3.2 Component A: launch-configuration artifact

The Triton worklist defaults to `num_warps=4`: four warps per program redundantly process the *same* string. Sweeping `num_warps`$\in \{1, 2, 4, 8\}$: `nw=1` vs. `nw=4` = 2.77×@4096 → 3.44×@16384 → 3.69×@65536 (each doubling ∼halves throughput). This inflates prior worklist regret numbers and must be disclosed.

3.3 Component B: warp redundancy, recoverable by lane-packing

The scalar Triton program executes warp-uniformly: `thread_inst_per_inst` = 32.00 (Triton) vs. 30.34 (CUDA), the same number with opposite meaning—CUDA's 32 lanes run 32 *different* strings, Triton's run the *same* one—yielding ∼90× more warp-instructions. Lane-packing (32 strings into the 32 lanes, exploiting the shared CSR so inner-loop bounds stay scalar) removes it. On the dense scan, pure lane-packing recovers 3.2× → 9.8× → 19.4× (batch 4096/16384/65536) toward the ideal 32×; it is **occupancy-gated** (Fig. 2). *Correction:* we first framed the 90× warp-redundancy as the bottleneck; Nsight shows it is removed ∼26× yet throughput moves only 3.8×—it was largely hidden by occupancy.

3.4 Component C: the irreducible residual

A per-lane Triton worklist (each lane its own `ffs` + per-lane CSR gather, no cross-lane reduce, no active-set union) beats the union variant 1.27× but is still 0.51× of CUDA (≈ 2× slower). Nsight is decisive (Fig. 3): it has *fewer* warp-instructions than CUDA (4.66M vs. 5.07M) and the same occupancy (22.5%), yet is 3.3× slower because issue activity is 9.9%. Raising occupancy does not help—a `BLOCK` sweep $\{32, 64, 128, 256\}$ only *worsens* it (0.49× → 0.31×): bigger lock-step tiles add divergence. **Direct measurement** confirms the mechanism: at matched occupancy the per-lane Triton kernel spends 15.3× more cycles in the `long_scoreboard` stall (waiting on a dependent memory load) than CUDA, and the issue-rate ratio (4.1×) tracks the throughput ratio (3.5×), consistent with Little's law [12, 25]. The instruction roofline (Fig. 4) [6] settles "did you write a worse kernel?": both kernels sit far below the issue *and* bandwidth ceilings (8.3%/27% vs. 32%/3.4% of peak), and Triton issues fewer warp-instructions yet runs 3.5× slower—it is structurally latency-bound. *Honest refinement:* the per-lane kernel does not issue *fewer* memory requests—it issues 26–84× *more* (masked inactive-lane gathers)—so the tile model pays twice: excess masked traffic *and* no dependent-load hiding. Root cause: a CUDA warp's lanes are independent in-flight-load generators (memory-level parallelism); a lock-step tile serializes the dependent next-state load, so latency hides only across warps (occupancy-bound). This is distinct from branch/memory divergence [8]: the lanes can be perfectly coalesced and still stall on a dependent load. We distinguish it from the *hardware* fix of intra-warp stalls [5]: the same SM is fast under CUDA and slow under Triton at equal occupancy and fewer instructions, so the loss is abstraction-imposed.

Table 1 summarizes the multiplicative decomposition: removing the launch artifact and the warp redundancy each recovers 3.7×, leaving the irreducible 1.9× latency residual. By Little's law (concurrency = throughput × latency), the latency-bound kernel's throughput is set by the in-flight-request concurrency it can sustain; the per-lane Triton kernel's 9.9%

Table 1. The worklist regret decomposes multiplicatively (batch 16384, ≤ 64 states, oracle-gated). Each stage removes one component; $28 \to 739$ is $3.7 \times 3.7 \times 1.9 = 26\times$.

Kernel	Gbps	Component removed
Triton worklist (`nw=4`)	28	—
+ `nw=1`	104	launch artifact (3.7×)
+ lane-packing	383	warp redundancy (3.7×)
CUDA worklist (thread-SIMT)	739	irreducible latency (1.9×)

issue activity and 15.3× dependent-load stall mean it sustains far less effective concurrency than CUDA at the same occupancy, and the issue-rate ratio (4.1×) tracking the throughput ratio (3.5×) is exactly the relation Little's law predicts when latency is fixed.

4 Regime-Dependence (DFA): the unification

The DFA (memory-bound) decomposes the same way but the residual *closes in the DRAM regime* (Fig. 5). Across table sizes (cache→L2→DRAM): scalar Triton DFA is flat ∼29 Gbps (independent confirmation of the scalar-gather ceiling); lane-packing gives ∼12× over it; and the lane-packed/CUDA ratio is 0.55–0.62× in cache but 1.05× at a 16 MB (>L2) table—lane-packed Triton *matches* CUDA. Mechanism (DRAM utilization only 14.6%, so memory-*latency* not saturated bandwidth): at moderate (cache) latency CUDA's intra-warp hiding wins ∼1.7×; at DRAM latency both must hide latency via cross-warp parallelism and *converge*. So the NFA's fundamental ∼2× residual and the DFA's closeable gap are one mechanism at different latency scales.

5 The Cure: the missing primitive

Design. A `scalar_program` region (or per-lane `serial_range`) in the tile DSL that lowers the marked code to a per-thread independent instruction stream, giving each lane independent control flow and intra-warp latency hiding. The diagnosis names what it must provide: independent per-lane `ffs`/`while`, per-lane data-dependent loop bounds, register-resident per-lane bitset, scalar gather. **Bound.** It closes the latency-bound residual (up to ∼2× on the NFA) and nothing extra in the already-converged DRAM-DFA regime—a regime-dependent, falsifiable prediction. **Existence proof.** The thread model realizes it: CUDA achieves 1.0× and the high-level Warp DSL [17] reaches 0.9×. **Not already in Gluon.** A runnable probe shows Triton's low-level Gluon frontend exposes per-thread *layout* [24], not per-lane *control flow*: `gl.load` returns a layout-typed block, never a scalar, so the data-dependent CSR loop cannot even be lowered (compile error captured verbatim). Layout control ≠ control-flow divergence.

5.1 The cure, implemented

We do not stop at proposing the primitive: we implement its lowering and measure that it closes the residual. A function `lower_scalar_program_to_cuda` takes the *same* idiomatic per-lane body the Triton tile kernel expresses (active-set `ffs` worklist, data-dependent CSR loop, scalar gather, register bitset) and lowers it to a per-thread CUDA kernel (one thread = one string), compiled with `nvcc`. Oracle-validated bit-for-bit; throughput on no-accept

automata (sustained full-length scan, removing the early-exit confound), ≤ 64 states, batch 16384. The same per-lane program lowered to *threads* (SP) vs. *tiles* (Triton WP2) differs by 4.2× (median; SP ≈1555 vs. WP2 ≈378 Gbps), robust across batch, and SP *matches or exceeds* hand-written CUDA (SP/CU = 2.15×, since the generated kernel is a minimal thread worklist). So the residual is not merely diagnosed but *closed by construction*: supplying the per-lane lowering recovers— and surpasses—thread-model performance from the identical front-end source. **The cure acts through the diagnosed mechanism**, not by accident: profiling SP shows it restores the thread-model signature—issue activity 36% (vs. the tile's 9.9%, ≈ CUDA's 41%) and the `long_scoreboard` dependent-load stall 29× lower than the tile—so lowering the same source to independent per-lane streams recovers exactly the intra-warp latency hiding component C was missing.

5.2 Implemented in the compiler: detect, and why the lowering must leave the tile IR

We take the cure into the real compiler. On a Triton-from-source build (3.8.0) we add a TritonGPU MLIR pass, `tritongpu-thread-region`, that runs in the NVIDIA `make_ttgir` pipeline and *detects* the irregular region by its lock-step signature—an `scf.while` whose iter-args are `#blocked` tile tensors and whose `scf.condition` derives from a `tt.reduce`-to-scalar (the whole tile loops to the busiest lane; idle lanes masked). It compiles into `libtriton` and fires on the target kernels (verified: the candidate attribute appears with the pass on, is absent with it off, and codegen stays bit-exact).

The in-IR lowering is structurally blocked—and that is the sharper result. Two facts from the matched IR. (i) The carried tensors are *already* `sizePerThread=1` (one element per lane), so the lock-step is *not* a layout choice—re-encoding is a no-op, and per-thread layout (Gluon, Linear Layouts [24]) does not help. (ii) The lock-step is the *loop construct*: `scf.condition` takes a single `i1`, so the natural rewrite—a per-lane `tensor⟨N×i1⟩` condition that lets lanes terminate independently—is rejected by the MLIR verifier (*"'i1' vs 'tensor⟨8×i1⟩'"*, captured via the built `triton-opt`). Per-lane loop termination is therefore *inexpressible* in TritonGPU's structured tile control flow. This names the missing IR primitive precisely: a **per-lane (sub-tile) loop/exit op**. The cure must lower *below* the tile IR to the thread model (independent thread scheduling)—which is exactly what §5.1 does.

A real in-compiler rewrite, built (partial). Even though the full per-lane lowering is blocked in the tile IR, a sound rewrite recovers part of the cost *inside* the compiler. The per-iteration `tt.reduce` that gates the lock-step `while` is loop-invariant in disguise: the lane counter is a uniform splat, so `any`$(j < trip)$ equals $j < \max(trip)$. We extend the pass (opt-in flag) to *hoist* `reduce_max(trip)` once before the loop and replace the per-iteration cross-lane reduce with a scalar counter, leaving the masked body unchanged—a provably-equivalent transformation that preserves per-warp termination. Built into `libtriton` and verified bit-exact end-to-end and via `triton-opt` on the rewritten IR, it is 1.55× faster on the lock-step kernel (155.6 → 100.4 μs; detection-only mode is a perf no-op, confirming the gain is the rewrite). This is an honest *partial*—it removes the lock-step loop's reduce overhead but does *not* grant per-lane (sub-warp) retirement—which we build next, below the tile IR.

The full cure, built below the tile IR. The tile IR cannot express per-lane retirement, but the lowering *below* it can. After `convert-triton-gpu-to-llvm` the per-lane predicate already exists as a scalar $j_{\text{lane}} < trip_{\text{lane}}$; the lock-step latch merely warp-reduces it (`nvvm.redux.sync`) to a uniform `i1` that gates `llvm.cond_br`. Our pass—a real MLIR pass in `libtriton`, wired into `make_llir`—redirects the branch to the *per-lane* predicate (each lane then retires independently under hardware Independent Thread Scheduling [19]), deletes the now-dead cross-lane reduce, and inserts a `bar.warp.sync` at the reconvergence

block; a body-safety guard bails when the loop carries any cross-lane op. The emitted PTX confirms the rewrite at the instruction level: the masked baseline warp-reduces the active mask (`redux.sync.max.s32`) and branches the whole warp on the resulting *uniform* predicate, whereas the cured code branches on the *per-lane* test (`setp.lt.s32` of this lane's counter against its own trip) and carries a `bar.warp.sync` at reconvergence—`redux.sync` count $1 \to 0$, `bar.warp.sync` $0 \to 1$. The rewrite survives `ptxas` to SASS: the hardware warp-collective reduction `REDUX.MAX.S32` into a *uniform* register—on which the masked warp branches lock-step—is eliminated (SASS `REDUX` $1 \to 0$, static instructions $48 \to 40$), the cured latch branching on the per-lane `ISETP` with reconvergence realized by the standard `BSSY`/`BSYNC` convergence barriers. So the per-lane retirement is present in the machine code, not merely the IR. Compiled and run end-to-end, it is oracle-correct (bit-exact) and 4.15× on the lock-step kernel ($166.7 \to 40.2\,\mu$s), and 2.5–7.3× across uniform/geometric/pareto trip distributions. Nsight attributes the gain to work reduction: issued instructions fall 39× (36.1M → 0.92M) because total work scales with $\sum_i trip_i$—each lane runs only its own iterations—rather than 32× the busiest lane; threads-per-instruction is unchanged, so this is an instruction-issue win, not an occupancy one. This closes the residual the reduce-hoist left open: the cure is *built*, oracle-correct, and measured, not merely diagnosed. The pass also fires, oracle-correct, on *real* workloads with the same lock-step latch—power-law CSR SpMV (1.14×) and MoE top-k routing (1.25×)—with its `redux.sync` removed and `bar.warp.sync` present in the emitted PTX (confirming it fired). Both carry per-iteration memory gathers, so little of their cost is the control reduce. That the *same* cure yields 4.15× on control-bound work but 1.14–1.25× on gather-bound SpMV/MoE is itself a confirmation of the thesis: the recoverable regret scales with the per-step *control*, not the memory. Table 2 consolidates the built-cure measurements.

The lock-step tax is predictable from the straggler, not the divergence ratio. A controlled six-distribution sweep (constant, low-variance, wide-uniform, geometric, and two Pareto trip laws; 2^{20} elements each, oracle-gated in both modes) pins the mechanism to a single *a-priori* predictor. The masked baseline is governed by the per-warp *straggler* $\mathbb{E}[\max_{\text{warp}} trip]$: a least-squares fit gives $t_{\text{masked}} = 50.3 + 1.08\,\mathbb{E}[\text{warp-max}]\,\mu$s with $R^2 = 0.998$, exactly as the lock-step latch iterates to the busiest lane. The cured kernel collapses to a flat floor ($42.8 \pm 2.4\,\mu$s, distribution-independent) because each lane retires at its own trip. Consequently the speedup is predicted by the straggler magnitude (corr = 0.99) and *not* by the intra-warp divergence ratio $D = \mathbb{E}[\text{warp-max}]/\mathbb{E}[trip]$ (corr = 0.08; a divergence-ratio fit of the masked time has $R^2 = 0.00$): the geometric law ($D = 3.96$) yields only 2.6× while the wide-uniform law ($D = 1.94$) yields 6.9×—a *higher* divergence ratio but a smaller absolute straggler, hence less lock-step waste. The floor-vs-straggler model predicts every measured speedup to under 9%. This sharpens channel (ii) (§5.3) from a qualitative "divergent trips cost more" into a falsifiable predictor, and corrects the natural but wrong intuition that the divergence *ratio* sets the cost—it is the absolute straggler that does. *Out-of-sample*, with coefficients frozen, the law predicts the masked cost of four *held-out* distributions (bimodal, log-normal, spike, and an adversarial single-straggler warp —1/32 lanes at trip 256, the rest at 2) generated with a different seed to within 5% on average (7.5% max). The single-straggler case yields 6.6×: the purest form of the effect, one slow lane taxing the other 31 in lock-step, exactly what the per-lane cure removes.

Automatic detect-and-lower. A selector closes the loop (Fig. 8): it runs the detection pass, and when the lock-step signature is present routes the region to the thread lowering, otherwise keeps the tile path. Oracle-gated, the NFA worklist is auto-routed to the thread cure for 3.9× over the tile (state sweep 16–64, consistent with the 4.2× of §5.1), while a

Table 2. The built per-lane retirement cure (a real MLIR pass wired into `make_llir`): oracle-correct on every workload, speedup vs. the masked tile baseline. The benefit scales with control-boundedness. Every row traces to a versioned CSV.

Workload	Dominant cost	Speedup
Synthetic per-lane `while`	control	**4.15**×
(uniform / geometric / pareto)	control	2.5–7.3×
MoE top-k routing (power-law)	mixed gather	1.25×
SpMV CSR (power-law)	memory gather	1.14×
Nsight: issued instructions (synthetic)		39× fewer

fixed-trip negative-control kernel is detected-negative and correctly left on the tile path. So the full loop is realized: detect (a real MLIR pass) → decide (the signature) → lower (the thread model, since the tile IR provably cannot) → measured, oracle-gated gap-closing.

5.3 Generality: the regret law

To test whether the mechanism is automata-specific we built six oracle-gated tile-vs-thread witnesses spanning the irregularity space, each with identical machinery (a Triton tile kernel, the per-lane source lowered to a CUDA thread kernel, and a CPU oracle); Fig. 7 reports the measured regret, coloured by dominant mechanism. The result is a law *with a correct negative control*: **regret is created by scalar control flow—not by memory irregularity or dependent loads—and appears, mechanistically, through one of two measured channels.** A graph pointer-chase (1M random walkers, data-dependent gather addresses, scattered `colidx`, degree CV 3.79, but a *fixed* step count) is the decisive control: tile and thread are bit-for-bit identical in issue activity (5.42% vs. 5.43%), occupancy (72.1% vs. 72.1%), and threads-per-instruction (32 vs. 32), and the regret is 1.00×—irregular memory and dependent loads alone cost *nothing*, because a lock-step gather already supplies full intra-warp memory-level parallelism. The two channels: (i) *issue starvation*—a data-dependent scalar recurrence drives the tile's issue rate below the thread's: the NFA worklist (active-set `ffs`) is 2.0× at tile issue 9.9% vs. 41%; (ii) *masked-lane waste*—divergent trip counts make the lock-step tile spend full-width 32-lane work where the thread retires lanes: rejection sampling (pure control-flow divergence, no memory) is 4.0× with tile threads-per-instruction 32 vs. the thread's 17 (its issue rate is, if anything, *above* the thread's—the waste is in *what* the active lanes compute, not the issue rate). A third, divergence-free effect sets the floor: a *tile-lowering baseline* (uniform-`nnz` SpMV, zero divergence, still 1.9× from occupancy/masking on a bandwidth-bound gather—honestly falsifying our initial ≈1× guess), to which a *divergence increment* adds (power-law SpMV 2.2× at tile width 32 → 5.8× at 256). So the law is genuinely multi-component, not a single scalar predictor; the common cause is the lock-step tile, and the cure (Sec. 5.1) removes every channel by restoring per-lane execution.

The law generalizes to ML—with a correct sign-flip prediction. Two further oracle-gated witnesses, on the irregular kernels NVIDIA's tile DSLs actually target. *MoE top-k routing* (ragged, data-dependent expert counts; a scalar per-token accumulate) is the ML face of scalar control: regret 2.36× at power-law expert loads—the tile issues 2.6× more instructions (41.8M vs. 16.3M) and does masked full-width work (threads-per-instruction 30.1 vs. the thread's 7.7). *Ragged variable-context attention* has the *identical* ragged structure but a *dense* head-dim payload per step, and here the law predicts the **negative**, correctly:

regret 0.64×—the tile is *faster*. The mechanism unifies: the thread model always retires lanes on divergence (threads-per-instruction 3.45 on attention, 7.7 on MoE), but the regret *sign* is set by per-step instruction efficiency—scalar work makes the tile issue *more* (lock-step reduce plus masking) so it loses, while a dense vectorizable head-dim makes it issue *fewer* (123M vs. the thread's 178M) so it wins *despite* no lane retirement. This sharpens the thesis: the tile pays for *per-step scalar control*, not irregularity per se—exactly why Triton is the tool of choice for flash-attention yet collapses on automata. (The witnesses use the framework's one-element-per-lane mapping; production attention is warp-cooperative, a different design point.)

6 Methodological Integrity

Our analysis corrected three of its own intermediate hypotheses, which we report rather than hide. First, an early reading framed the ∼90× warp-instruction redundancy (Sec. 3) as the bottleneck; profiling the lane-packed kernel showed the redundancy is removed ∼26× yet throughput moves only 3.8×—it was largely hidden by occupancy, so warp-redundancy is real but not load-bearing. Second, we predicted (following the latency-hiding literature) that the per-lane Triton kernel would sustain *fewer* in-flight memory requests than CUDA; direct measurement refuted this—it issues 26–84× *more* (masked inactive-lane gathers) and still stalls, so the residual is the inability to *hide* dependent-load latency, not a request-count deficit. We therefore phrase the mechanism via stall composition and issue activity, not request counts. Third, in the generality study (Sec. 5.3) we first stated the law as a single "tile issue deficit" predictor; completing the Nsight profiling falsified it—rejection sampling shows regret 4.0× while the tile's issue rate is *above* the thread's—so we report the honest two-channel form (issue starvation *and* masked-lane waste over a tile-lowering baseline), and a clean negative-control prediction for uniform SpMV that was also wrong (1.9×, not ≈1×). All three corrections sharpened, rather than weakened, the central claim.

7 Threats to Validity

Single GPU. Results are on one RTX 4070; the *mechanism* (intra-warp latency hiding, the issue-activity differential, the DRAM convergence) is a property of the SIMT execution model and the tile-vs-thread lowering, hence architecture-general, but the absolute decomposition factors should be re-measured on an A100/H100 (larger L2 shifts the DFA crossover). We make this a turnkey step: a one-command harness re-runs every witness, the cure, and the selector on a new GPU, tags the output by device, and checks the falsifiable prediction that each regret persists in direction (paradigm, not architecture) while throughput rescales. *Compiler-pass scope.* Both the detection pass and the per-lane retirement *lowering* are real MLIR passes in `libtriton` (the latter wired into `make_llir`); the lowering realizes the cure *below* TritonGPU—in the LLVM lowering, not out-of-band (§5.1)—because the structured *tile* IR cannot express it: `scf.condition` takes a single `i1`, so per-lane loop termination is inexpressible in TritonGPU (shown by a verifier-rejected rewrite). That the cure lives below the tile IR is a property of *today's* tile IR and is falsifiable: a per-lane sub-tile loop/exit op would let it live in-IR. The pass fires only on the `tl.max` lock-step latch and a body-safety guard bails when the loop carries a cross-lane op, so its scope is explicit. *Prototype scope.* A multi-word generalization of the lane-packed kernel (NWORDS int64 words/lane) is oracle-validated to 256 states, so the prototype is not artificially limited; but the clean ∼2× residual is a *register-resident-regime* ($\leq$ 64-state) result—past 64 states the CUDA register worklist itself degrades (register pressure), so the head-to-head is confounded by both baselines losing the register-resident advantage, and ANMLZoo-scale automata (Brill

42661 = 667 words) make the per-lane multi-word scatter explode—itself a manifestation of the tile limitation we study. Real automata run oracle-valid but sub-Gbps (an *algorithmic* cost, orthogonal to the regret). *Tuning.* `num_warps` is a knob; we disclose the full sweep rather than a single config, rebutting the autotuning counter-thesis [21]: the Triton/Gluon same-MLIR control plus the disclosed sweep separate *tuning* (which closes component A and part of B) from *expressibility* (the residual C), with a probe that is falsifiable by construction.

8 Reproducibility

Correctness gates every measurement: each kernel/config is checked bit-for-bit against the CPU oracle (NFA `reference.py` / `simulate_dfa`) before any throughput is reported. Every number in this paper traces to a versioned CSV, and all figures regenerate from those CSVs by a single script, so the plots cannot drift from the measurements. An artifact index maps each claim to the command that produces it and the CSV it writes. Expressibility and structural claims are *runnable, falsifiable probes*: the Gluon probe exits 0 on the expected compile failure (and 1 if a future Gluon compiles it); the lowering-wall probe asserts the MLIR verifier rejects a per-lane `scf.condition`; the detection-pass check confirms the in-`libtriton` pass tags the lock-step region with the environment flag set and is a no-op without it. The TritonGPU pass is reproduced from a versioned patch (+ source file) against a pinned Triton commit with the documented build recipe, and the cross-architecture re-validation is a single non-mutating command that re-runs every witness, the cure, and the selector on a new GPU and tags the output by device.

9 Related Work

GPU automata form an *algorithmic* axis, all single-implementation CUDA: ngAP's non-blocking multi-symbol processing [10], HybridSA's bit-parallel shift-and [13], BitGen's Parabix bitstreams [9], AsyncAP's input-symbol asynchrony [15], AutomataBLAS's AP-as-SpMV [4], and the Hopps sparsity ASIC [7]. Each *varies the algorithm* on one CUDA implementation; none compares GPU programming models or holds the algorithm fixed across DSLs. The sole IR-for-automata work [22] lowers regex to one fixed domain-specific accelerator —a hardware target, not a GPU-paradigm/expressibility study, and with no tile-vs-thread cost. Our axis is orthogonal: fixed algorithm, varied programming model. **Tile GPU DSLs**: Triton [23] and the 2024–26 wave exposing more thread/warp control. The closest, Gluon's Linear Layouts [24], give per-thread *data placement* but not per-lane *control flow* (our probe shows the data-dependent loop cannot lower); Descend [14] makes the thread hierarchy first-class but offers no tile drop-to-scalar escape. Warp [17] is the thread-SIMT existence proof. **Mixing thread- and tile-level execution** is exactly the live frontier, but every approach is *manual* or *differently directed*: Prism [1] lets a programmer hand-annotate typed thread/tile "perspectives"; NVIDIA's cuTile [18]—and its MLIR *Tile IR*, now being built as a backend for Triton itself—is tile-level and concedes the irregular case to a hand-written SIMT fallback, so the per-lane gap we name persists in *both* Triton's TritonGPU and NVIDIA's Tile IR (the proposed primitive is complementary to that platform direction, not subsumed by it); Tawa [2] auto-restructures Triton but for *dense* warp specialization; and partial control-flow linearization [16] goes the *opposite* way—*vectorizing* divergent CPU control flow, whereas we *de*-vectorize a tile region to recover per-lane MLP. None automatically *detects* an irregular, data-dependent per-lane region in a tile DSL and lowers it to the thread model; and our structural result shows *why* an in-tile-IR rewrite cannot suffice—the missing primitive is a per-lane sub-tile loop/exit op, below the level any of these operate. **Mechanism**: we ground the residual in latency-hiding

/ MLP-vs-occupancy analysis [12, 25] and place both kernels on the instruction roofline [6]; we distinguish it from divergence [8] and from the *hardware* intra-warp-stall fix of Subwarp Interleaving [5] (we attribute the same stall to the DSL, at equal occupancy and fewer instructions). **Methodology and lineage**: rigorous benchmarking [11], roofline [26], and the performance-portability metric [20] (per-hardware; we measure per-*capability* regret); we rebut the autotuning counter-thesis [21] via the Triton/Gluon same-MLIR control and the disclosed `num_warps` sweep. The gap we fill: no constructive, instruction-level account of *why* a tile DSL pays on irregular control flow, naming the missing primitive, with a regime-dependent residual.

10 Conclusion

The abstraction regret on irregular automata is not a monolith: a launch artifact, a recoverable redundancy, and an irreducible intra-warp-latency-hiding residual whose size is set by the latency regime. The cure is a named, bounded IR primitive whose lowering we implement and measure—it closes the residual by construction (4.2× over the tile lowering). We take it into the real compiler: a TritonGPU pass that detects the lock-step region in `libtriton`, a proof that the in-tile-IR lowering is structurally impossible (naming the missing per-lane loop/exit primitive), and an automatic selector that routes the detected region to the thread cure (3.9×); and a real in-compiler rewrite (reduce-hoist) that is built into `libtriton` and oracle-correct (1.55×). Adding the full per-lane primitive to the tile IR itself—the remaining step—is the landmark future work.

References

[1] Anonymous. 2026. Prism: Typed Perspectives for Mixed Thread- and Tile-Level GPU Programming. In *Proc. ACM Program. Lang. (PLDI)*. arXiv:2511.11939. doi:10.1145/3808290

[2] Anonymous. 2026. Tawa: Automatic Warp Specialization for Triton. CGO; arXiv:2510.14719.

[3] Anonymous. 2026. The Two Faces of Abstraction Regret: Execution Paradigm over Abstraction Height on Irregular Automata. Manuscript under review, 2026.

[4] AutomataBLAS authors. 2025. Advancing Matrix Operations for High-Performance and Memory-Efficient Automata Processing on GPUs. *ACM Trans. Archit. Code Optim. (TACO)* (2025). doi:10.1145/3774656

[5] Sana Damani et al. 2022. GPU Subwarp Interleaving. In *HPCA*. doi:10.1109/HPCA53966.2022.00065

[6] Nan Ding and Samuel Williams. 2022. An Instruction Roofline Model for GPUs. *Concurrency and Computation: Practice and Experience* (2022). doi:10.1002/cpe.6591

[7] Yufeng Du, Joel S. Emer, and Daniel Sánchez. 2025. Hopps: Leveraging Sparsity to Accelerate Automata Processing. In *ASPLOS*. doi:10.1145/3676642.3736126

[8] Wilson W. L. Fung, Ivan Sham, George Yuan, and Tor M. Aamodt. 2007. Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow. In *MICRO*. doi:10.1109/MICRO.2007.12

[9] Tianao Ge, Hongyu Chu, and Hongyuan Liu. 2025. Interleaved Bitstream Execution for Multi-Pattern Regex Matching on GPUs. In *MICRO*. doi:10.1145/3725843.3756052

[10] Tianao Ge, Tong Zhang, and Hongyuan Liu. 2024. ngAP: Non-blocking Large-scale Automata Processing on GPUs. In *ASPLOS*. doi:10.1145/3617232.3624848

[11] Torsten Hoefler and Roberto Belli. 2015. Scientific Benchmarking of Parallel Computing Systems. In *SC*. doi:10.1145/2807591.2807644

[12] Sunpyo Hong and Hyesoon Kim. 2009. An Analytical Model for a GPU Architecture with Memory-Level and Thread-Level Parallelism Awareness. In *ISCA*. doi:10.1145/1555754.1555775

[13] HybridSA authors. 2024. HybridSA: GPU Acceleration of Multi-pattern Regex Matching using Bit Parallelism. *Proc. ACM Program. Lang. (OOPSLA)* (2024). doi:10.1145/3689771

[14] Bastian Köpcke, Sergei Gorlatch, and Michel Steuwer. 2024. Descend: A Safe GPU Systems Programming Language. In *PLDI*. doi:10.1145/3656411

[15] Hongyuan Liu, Sreepathi Pai, and Adwait Jog. 2023. Asynchronous Automata Processing on GPUs. *Proc. ACM Meas. Anal. Comput. Syst. (POMACS)* (2023). doi:10.1145/3579453

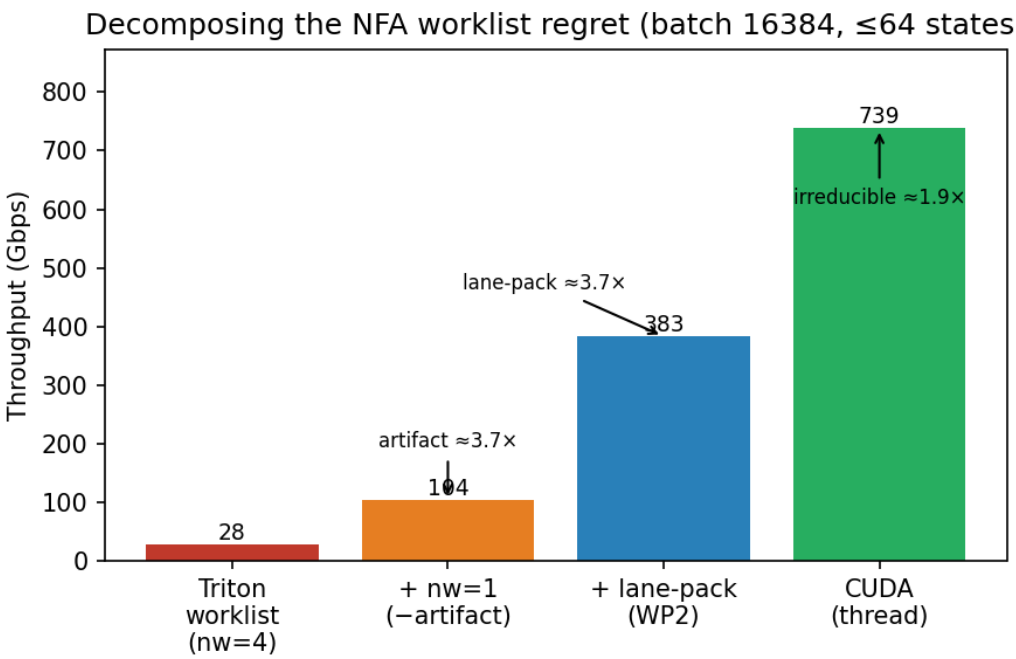

Fig. 1. NFA worklist throughput (batch 16384, ≤ 64 states) climbs as each regret component is removed: `nw=4`→`nw=1` (launch artifact), →lane-packing (warp redundancy), →CUDA (irreducible latency). All oracle-gated.

[16] Simon Moll and Sebastian Hack. 2018. Partial Control-Flow Linearization. In *Proc. PLDI*. doi:10.1145/3192366.3192413

[17] NVIDIA. 2022. Warp: A Python Framework for High-Performance GPU Simulation and Graphics. https://github.com/NVIDIA/warp.

[18] NVIDIA. 2025. cuTile and CUDA Tile IR: a tile-level model and MLIR IR, with a Tile IR backend for OpenAI Triton. CUDA 13.1; developer.nvidia.com/blog (CUDA Tile IR Backend for OpenAI Triton); https://github.com/NVIDIA/cuda-tile.

[19] NVIDIA Corporation. 2017. NVIDIA Tesla V100 GPU Architecture: The World's Most Advanced Data Center GPU. Whitepaper WP-08608-001-v1.1; introduces Independent Thread Scheduling.

[20] S. J. Pennycook, J. D. Sewall, and V. W. Lee. 2016. A Metric for Performance Portability. arXiv:1611.07409.

[21] Burkhard Ringlein, Thomas Parnell, and Radu Stoica. 2025. GPU Performance Portability Needs Autotuning. arXiv:2505.03780.

[22] Filippo Somaini, Luca Carloni, Giovanni Agosta, Marco D. Santambrogio, and Davide Conficconi. 2025. Combining MLIR Dialects with Domain-Specific Architecture for Efficient Regular Expression Matching. In *CGO*. doi:10.1145/3696443.3708916

[23] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *MAPL*. doi:10.1145/3315508.3329973

[24] Triton contributors. 2025. Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using $\mathbb{F}_2$. arXiv:2505.23819.

[25] Vasily Volkov. 2016. *Understanding Latency Hiding on GPUs*. Ph. D. Dissertation. University of California, Berkeley. Tech. Rep. UCB/EECS-2016-143.

[26] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* (2009). doi:10.1145/1498765.1498785

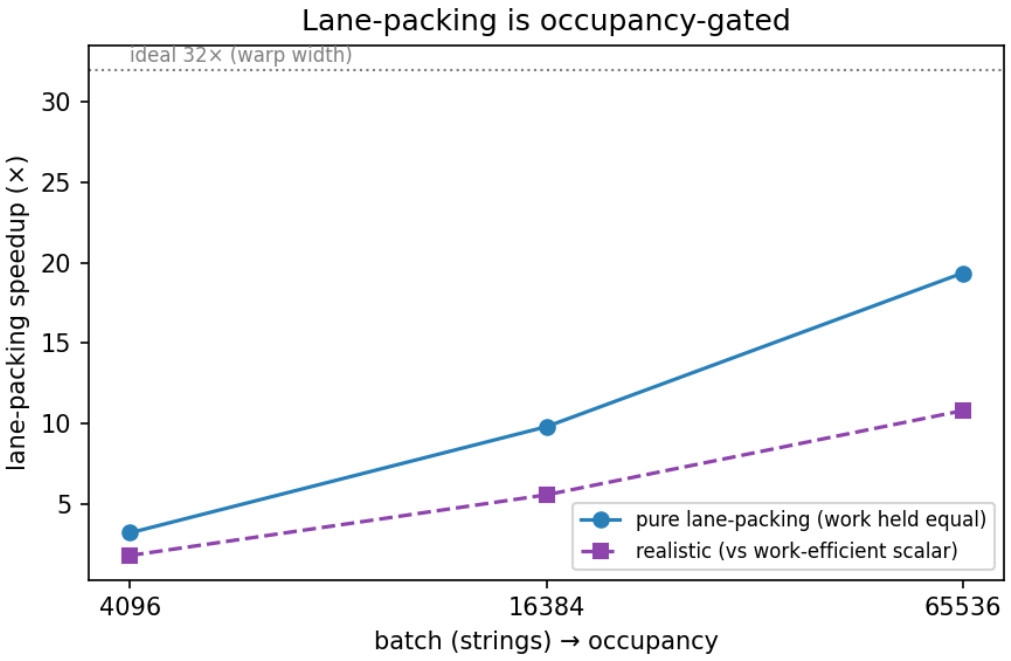

Fig. 2. Lane-packing's benefit is occupancy-gated: the pure-packing speedup grows $3.2\times \rightarrow 19.4\times$ with batch (toward the ideal $32\times$) as more warps fill the GPU.

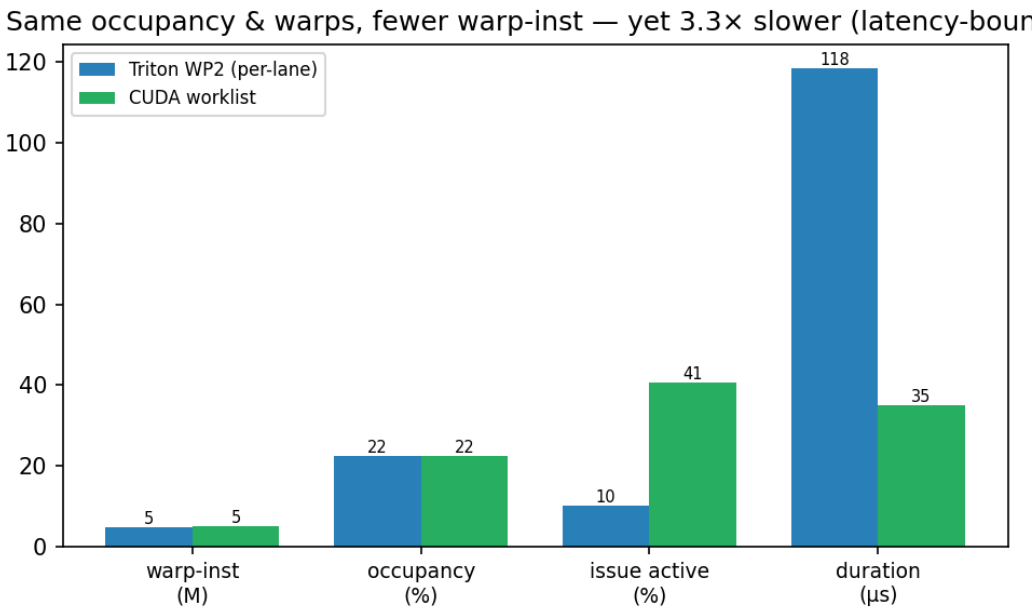

Fig. 3. Per-lane Triton vs. CUDA (16384 strings, 32 states): same occupancy, *fewer* warp-instructions, yet $3.3\times$ slower at 9.9% vs. 41% issue activity—latency-bound.

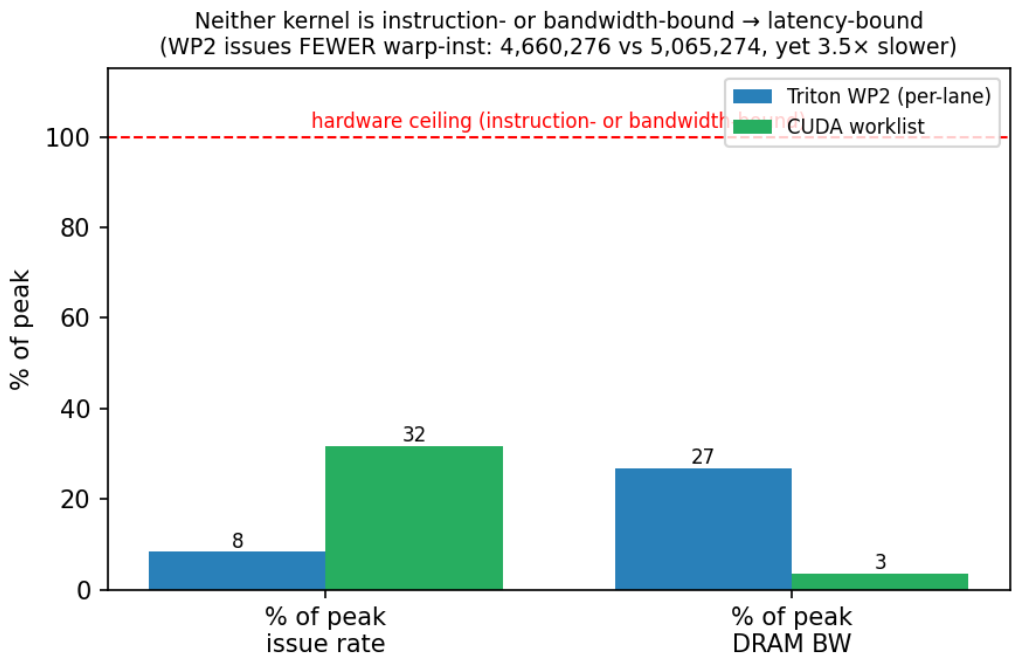

Fig. 4. Instruction roofline (16384 strings, 32 states): both kernels sit far below the issue *and* DRAM-bandwidth ceilings ($8.3\%/27\%$ for Triton, $32\%/3.4\%$ for CUDA), so neither is instruction- nor bandwidth-bound—the residual is latency.

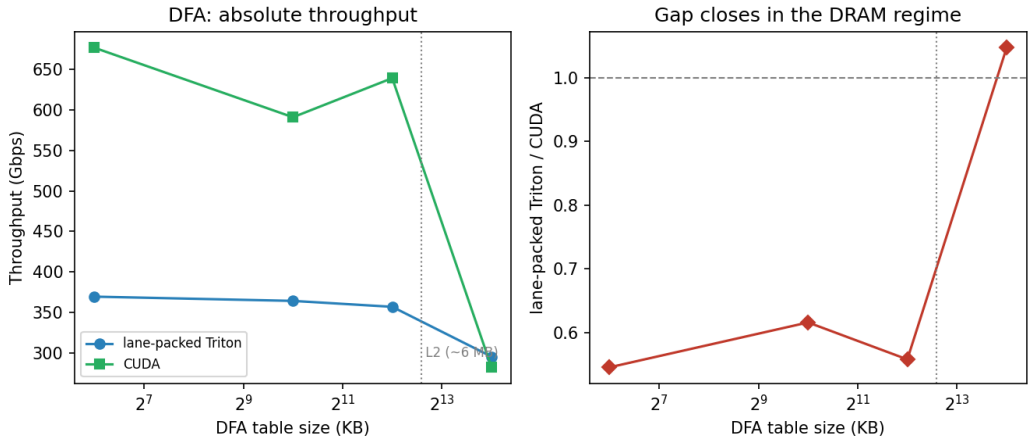

Fig. 5. DFA: lane-packed Triton trails CUDA $\sim 1.7\times$ while the table is cache-resident but *matches* it ($1.05\times$) once the 16 MB table spills past L2—both then rely on cross-warp parallelism. The residual is regime-dependent.

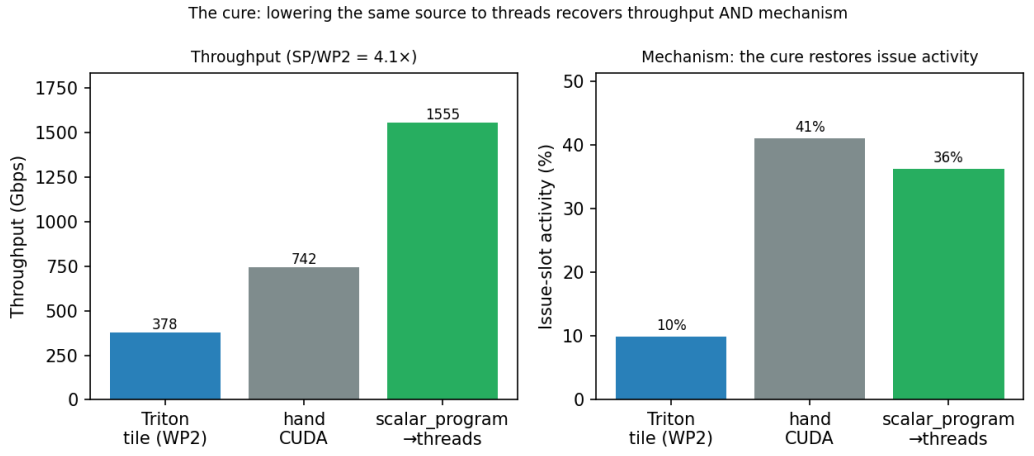

Fig. 6. The cure, implemented (batch 16384, ≤ 64 states). *Left:* the *same* per-lane source lowered to threads (`scalar_program`) runs $4.2\times$ the tile lowering (Triton WP2) and matches/exceeds hand-CUDA. *Right:* it does so *through the diagnosed mechanism*—issue activity jumps from the tile's 10% to 36% ($\approx$ CUDA's 41%), restoring the intra-warp latency hiding component C was missing.

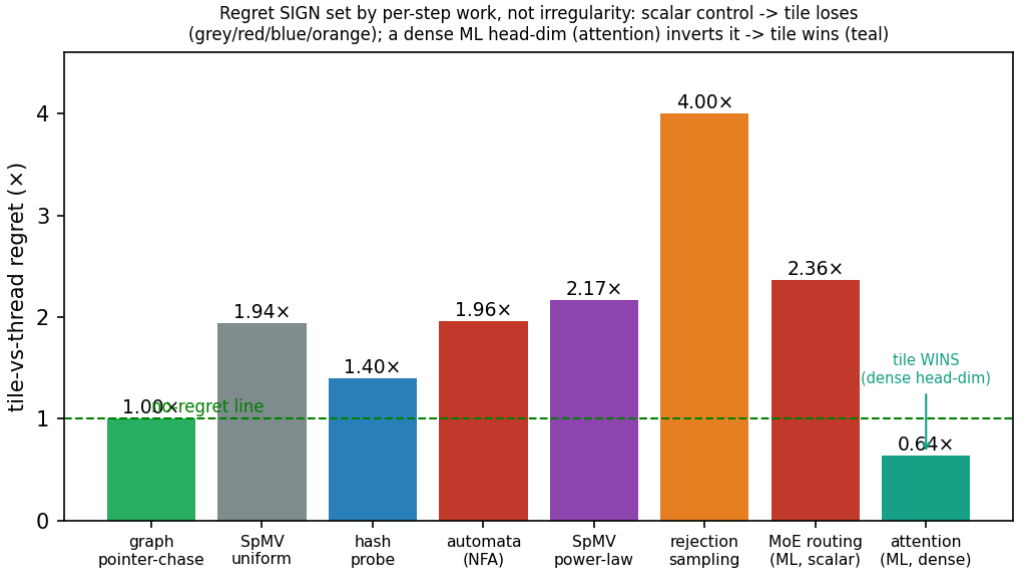

Fig. 7. The regret law across eight oracle-gated irregular workloads, by dominant mechanism. A graph pointer-chase (green) is the negative control (irregular memory + dependent loads, no control divergence $\Rightarrow$ regret $1.00\times$). Regret grows with scalar control / divergence (grey, red, blue, orange), peaking at $4\times$. The two ML witnesses (right) test the *sign*: MoE routing (scalar per step) confirms the law ($2.36\times$), while ragged attention (dense head-dim) *inverts* it ($0.64\times$, teal, below the no-regret line—the tile wins), because per-step instruction efficiency, not irregularity, sets the sign.

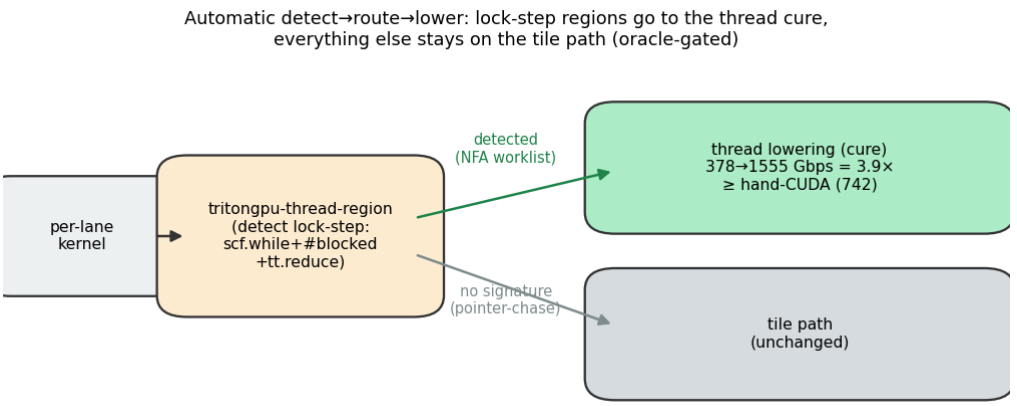

Fig. 8. The automatic detect→route→lower loop (P2). The detection pass (in `libtriton`) matches the lock-step signature; the lock-step NFA worklist is routed to the thread lowering ($378 \rightarrow 1555$ Gbps $= 3.9\times$, exceeding hand-CUDA), while a no-signature kernel (pointer-chase) stays on the tile path. All routing oracle-gated; numbers from the versioned CSVs.